CSCI 4177/5709 | ASSIGNMENT 3

StudyFlow

Performance, Security, and Monitoring Report

FINAL SUBMISSION

Verified against the assignment instructions and rubric. Includes reproducible load tests, four measured optimizations, security remediation, and live monitoring evidence.

Prepared by

Arup Halder

Course and term

CSCI 4177/5709, Summer 2026

Repository

github.com/beaprogram/studyflow-assignment3

Report date: July 15, 2026

OUTCOME

1. Executive Summary

StudyFlow was profiled under light and moderate load, optimized on both the client and server, rescanned for security weaknesses, and instrumented for continuous operational visibility. All required implementation and evidence artifacts are in the linked repository.

Measured outcome

Overall average latency improved by 42.3% under light load and 47.0% under moderate load. p95 latency improved by 46.2% and 51.3%, respectively, while all 4,808 submitted JMeter samples completed with a 0.00% error rate.

Rubric coverage

Rubric area	Completed evidence	Status
Baseline testing	JMX plus light/moderate baseline CSVs and bottleneck analysis	Complete
Client optimization	Route code splitting and memoized course rendering	Complete
Server optimization	TTL response cache and compound MongoDB index	Complete
Performance comparison	Same plan rerun; avg, p95, RPS, errors, and percentages	Complete
Security	ZAP before/after reports and two remediations	Complete
Monitoring	Prometheus metrics and six-panel Grafana dashboard	Complete
Documentation	This report, repository instructions, and evidence inventory	Complete

Submission at a glance

- Repository: https://github.com/beaprogram/studyflow-assignment3
- Primary Brightspace file: StudyFlow_Assignment3_Report.pdf
- Editable backup: StudyFlow_Assignment3_Report.docx

2. Baseline Test Method

The baseline and optimized measurements use the same parameterized JMeter sampler tree. A setup thread logs in once and extracts the bearer token. The load group then requests the authenticated course list, index page, JavaScript entry bundle, and CSS bundle with a 500 ms Constant Timer. This preserves an identical workload before and after.

Load scenarios

Scenario	Virtual users	Ramp-up	Loops/user	Samples/run
Light	10	30 seconds	10	401
Moderate	50	60 seconds	10	2,001

Metrics captured

- Average response time and p95 latency for each sampler and for the total run.
- Throughput in requests per second and request error rate.
- Raw timestamp, elapsed time, status, bytes, latency, connect time, URL, and thread counts in CSV.

Three baseline bottlenecks

Bottleneck	Baseline evidence	Why it mattered
POST Login	324 ms light; 296 ms moderate	Slowest single request; includes authentication and database work.
GET Courses	77.6/78.6 ms avg; 86/87 ms p95	Repeated authenticated query dominated latency during the load group.
Main JS bundle	578,126 bytes	Large initial payload contained charting code not needed on first render.

Interpretation note

Login runs once during setup, so it was identified as a bottleneck but was not the focus of the repeated read-path optimization. The course endpoint and first-load bundle offered the highest impact within the assignment scope.

3. Client-Side Optimizations

3.1 Route-level code splitting

The Analytics page imports Recharts, the heaviest client dependency. It was changed from an eager import to React.lazy with a dynamic import and Suspense fallback. The browser now downloads chart code only when the user navigates to Analytics.

```
const Analytics = lazy(() => import('./pages/Analytics.jsx'))
```

Bundle metric	Baseline	Optimized	Improvement
Initial JavaScript entry	578,126 bytes	177,350 bytes	69.3% smaller

The optimized build emits the Analytics/Recharts code as a separate deferred chunk. This reduces initial network transfer, parsing, and evaluation work for users who do not immediately open the analytics route.

3.2 Memoized course list and derived data

CourseRow now uses React.memo, summary calculations and the row collection use useMemo, and the loading callback uses useCallback. Stable course data no longer causes every row to render after unrelated parent-state changes.

```
const CourseRow = memo(function CourseRow({ course }) { ... })
```

Deterministic benchmark	Baseline	Optimized	Improvement
20 rows + 50 unrelated updates	1,020 renders	20 renders	98.0% fewer

The benchmark is included at client/benchmarks/memoization-benchmark.mjs and runs with npm run benchmark:memo. It isolates render behavior, so the result is reproducible without relying on device-specific browser timing.

Client impact

The first optimization reduces bytes delivered on initial navigation; the second reduces repeated rendering after the page is loaded. Together they address network, JavaScript evaluation, and React reconciliation costs.

4. Server-Side Optimizations

4.1 Short-lived course-list cache

Read-heavy course-list payloads are cached in process memory for 30 seconds. Keys include the authenticated user, page, limit, and term. Course create, update, and delete operations immediately invalidate that user's list entries, limiting stale data risk.

```
const key = `courses:list:${userId}:${page}:${limit}:${term}`;
cache.set(key, payload, 30 * 1000);
```

GET Courses	Baseline avg	Optimized avg	Improvement	Errors
Light	77.6 ms	40.0 ms	48.5%	0.00%
Moderate	78.6 ms	38.3 ms	51.3%	0.00%

An X-Cache response header exposes HIT, MISS, or DISABLED, making the behavior easy to verify. CACHE_ENABLED=false preserves a controlled baseline path.

4.2 Compound query-and-sort index

The course-list query filters by owner and sorts by createdAt descending. The model now defines { owner: 1, createdAt: -1 }, allowing MongoDB to return records in index order instead of scanning and sorting the collection.

Explain metric	Collection scan	Compound index	Improvement
Plan	SORT <- COLLSCAN	LIMIT <- FETCH <- IXSCAN	No in-memory sort
Documents examined	10,006	20	99.8% fewer
Execution time	11 ms	1 ms	90.9% faster

Why both changes are needed

The index improves unavoidable database reads and cold-cache requests. The cache removes repeated database work during short bursts. Write-triggered invalidation keeps both performance and correctness in balance.

5. Before-and-After Comparison

The optimized build was rerun with the same JMX plan, user counts, ramp times, loops, sampler tree, and think time. Percentage improvement for latency is calculated as (baseline - optimized) / baseline x 100.

Overall results

Scenario / metric	Baseline	Optimized	Change
Light average latency	22.6 ms	13.0 ms	42.3% better
Light p95 latency	80 ms	43 ms	46.2% better
Light throughput	8.25 req/s	8.32 req/s	+0.9%
Light error rate	0.00%	0.00%	No errors
Moderate average latency	21.7 ms	11.5 ms	47.0% better
Moderate p95 latency	78 ms	38 ms	51.3% better
Moderate throughput	24.92 req/s	25.03 req/s	+0.4%
Moderate error rate	0.00%	0.00%	No errors

Endpoint-level results

Scenario / endpoint	Avg before	Avg after	p95 before	p95 after	Avg gain
Light - GET Courses	77.6	40.0	86	47	48.5%
Light - JS bundle	4.2	3.3	6	5	21.9%
Moderate - GET Courses	78.6	38.3	87	44	51.3%
Moderate - JS bundle	3.6	2.7	5	4	25.9%

Units: All endpoint average and p95 values are milliseconds.

Result interpretation

Latency improved substantially while throughput rose slightly. Because each scenario uses fixed loops and a 500 ms timer, throughput is primarily workload-bound rather than an open-ended saturation measure. The unchanged 0.00% error rate confirms stability.

6. OWASP ZAP Security Validation

A headless ZAP baseline scan was captured before remediation and repeated against the optimized local production preview after the fixes. Both HTML and JSON reports are stored under zap/.

The headless ZAP baseline scan reported 9 alerts: 0 high, 2 medium, 5 low, and 2 informational. Both medium-severity findings are remediated below with configuration snippets. The five low-severity alerts (Cross-Origin-Embedder-Policy, Cross-Origin-Opener-Policy, Cross-Origin-Resource-Policy, Permissions-Policy, and X-Content-Type-Options headers) are addressed by the same security-header changes as defense in depth. The informational alerts require no remediation.

Baseline findings and fixes

Baseline alert	Risk / count	Remediation	Final result
CSP Header Not Set	Medium / 3	Restrictive Content-Security-Policy across Vite, Netlify, and serve	Resolved
Missing Anti-clickjacking Header	Medium / 1	frame-ancestors 'none' plus X-Frame-Options: DENY	Resolved

Remediation detail

```
default-src 'self'; script-src 'self'; style-src 'self'; img-src 'self' data:;
connect-src 'self'; object-src 'none'; base-uri 'self'; form-action 'self';
frame-ancestors 'none'
```

- Removed the only inline React style, so style-src does not require 'unsafe-inline'.
- Added X-Frame-Options: DENY and frame-ancestors 'none' for legacy and modern clickjacking protection.
- Added nosniff, no-referrer, Permissions-Policy, and cross-origin isolation headers as defense in depth.

Final scan

0 high-risk and 0 medium-risk findings

The July 15, 2026 after-scan crawled six URLs. Its only remaining alerts are informational: Modern Web Application and Storable but Non-Cacheable Content. The two required medium findings are absent from the final JSON and HTML reports. The content alert changed from 'Storable and Cacheable' to 'Storable but Non-Cacheable' because the new Cache-Control headers took effect.

Evidence files: zap/zap-report-before-web.html, zap/zap-report-before-web.json, zap/zap-report-after-web.html, zap/zap-report-after-web.json, and zap/remediation.md.

7. Prometheus and Grafana Monitoring

The API exposes Prometheus metrics at /metrics using prom-client. Docker Compose runs Prometheus and Grafana; provisioning files connect the data source and load the StudyFlow dashboard automatically. The dashboard was verified with live JMeter traffic.

Required signal	Implementation / panel
CPU	Node process user and system CPU rate
Memory	Resident memory and V8 heap used
Response time	p50, p95, and p99 from the HTTP duration histogram
Error rate	4xx/5xx share derived from request counters
Additional	Throughput by route and requests in flight

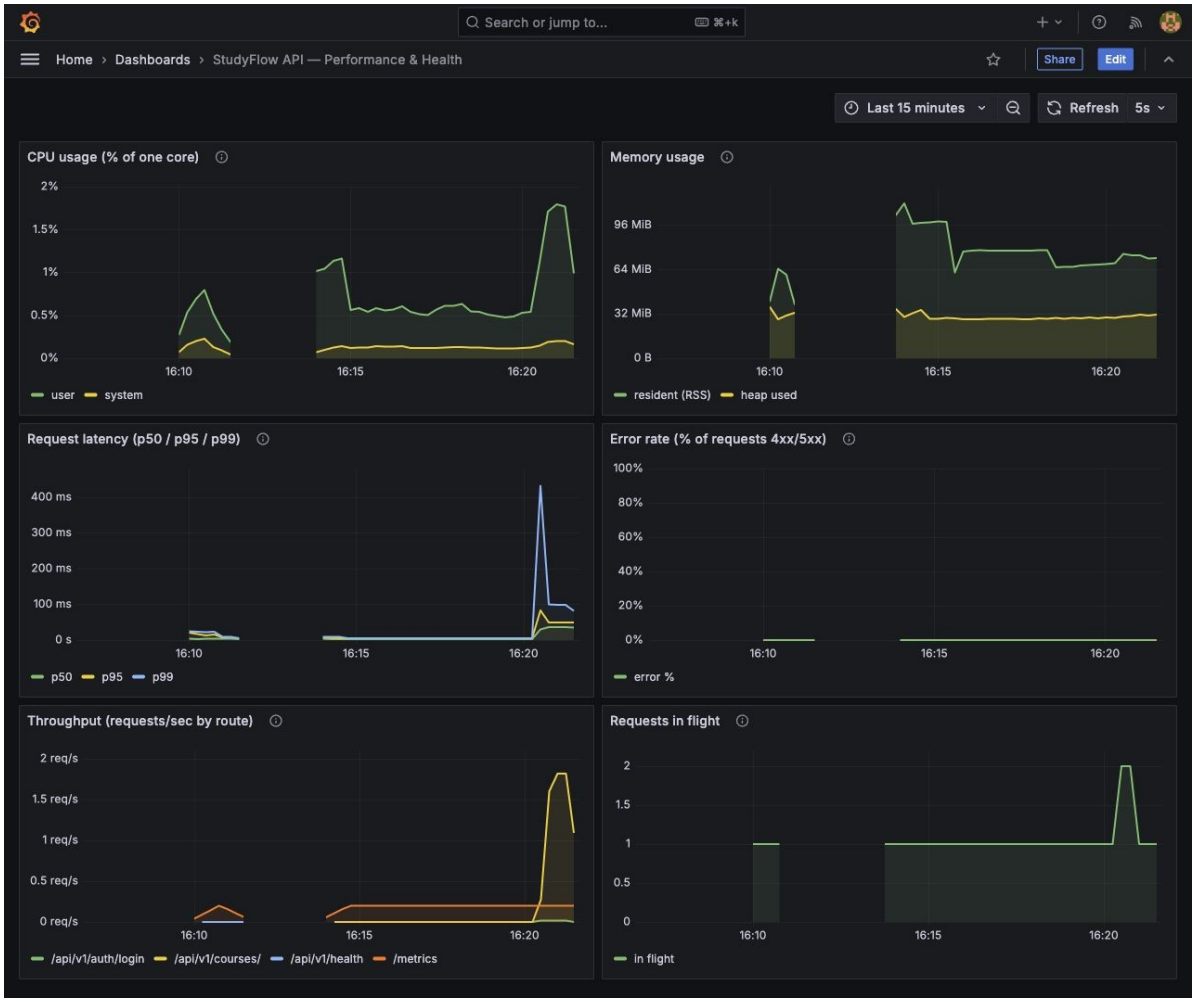

Figure 1. StudyFlow Grafana dashboard under live API load.

8. Validation and Reproducibility

Final validation status

Check	Result	Evidence
API smoke tests	6 of 6 passing	api/tests/smoke.test.js
Client production build	Passing	Vite optimized build
Production dependency audit	0 vulnerabilities	npm audit --omit=dev
Memoization benchmark	98.0% fewer row renders	client/benchmarks/
JMeter runs	4,808 samples; 0.00% errors	Four CSV result files
MongoDB explain	IXSCAN; 20 docs vs 10,006	api/seed/explain-courses.js
OWASP ZAP	0 high; 0 medium	Final JSON/HTML reports
Monitoring	Prometheus up; six panels	Dashboard JSON and screenshot

Core reproduction commands

```
cd api && npm test
cd client && npm run build && npm run benchmark:memo
cd jmeter && python3 compare.py
cd monitoring && docker compose up -d
```

Repository deliverable inventory

- client/ - optimized React/Vite source, benchmark, and security-header configuration.
- api/ - optimized Express/MongoDB source, tests, index evidence, cache, and metrics.
- jmeter/ - local and deployment JMX plans plus baseline/optimized light/moderate CSVs.
- zap/ - before/after reports, scan configuration, and remediation evidence.
- monitoring/ - Compose, Prometheus, Grafana provisioning, dashboard, and screenshot.
- deliverables/ - this report in PDF and Word plus the Brightspace checklist.

9. Conclusion

StudyFlow now has a smaller initial client bundle, fewer unnecessary React renders, a faster indexed and cached read path, stronger browser security headers, and continuous operational monitoring. The same JMeter workload demonstrates meaningful latency gains without introducing errors.

Assignment status: complete

Every rubric area has implementation evidence, measured results, and a reproducible path in the repository. The final report is ready for Brightspace after the completed repository is pushed to the GitHub URL below.

Repository

GitHub repository - StudyFlow Assignment 3

Primary Brightspace deliverable

StudyFlow_Assignment3_Report.pdf

The Word file is retained as an editable backup. Source artifacts are submitted through the repository link in this report; secrets, node_modules, and generated build folders are excluded.

Evidence index

Question	Go to
How was load tested?	Section 2 and jmeter/
What changed on the client?	Section 3 and client/
What changed on the server?	Section 4 and api/
What improved?	Section 5 and PERFORMANCE_EVIDENCE.md
Were security findings fixed?	Section 6 and zap/
How is the system monitored?	Section 7 and monitoring/